GIT RECOVERY · BROWSER LOGIN · LIVE CONFLICT FIX · YOUR OWN AGENT · CTF

Git + your own agent.

Last time half of us got stuck on login. Today we fix everyone to green — browser login, two connect paths, live merge conflict — then make an AI edit your page and push it back. 3:30 help desk, 4 PM main session.

TONIGHT'S RUN-OF-SHOW

Three blocks. All hands-on.

PART 1 / GIT FIX-IT

Everyone to green

Browser login (no tokens), Path A online → connect, Path B init → push, clone → branch → live conflict fix.

PART 2 / AGENT · LIVE

Your own agent edits your page

VS Code + Kilo Code extension, give it context, free Gemini key, push the upgrade.

PART 3 / CTF

Flag-hunting continues

Jagath takes over. Stego round 2 — same workflow, harder images.

BEFORE WE START — 2 MINUTES

If any box is unticked, fix it now.

☐ 1

Laptop + charger

You will type in every block. No spectators tonight.

☐ 2

VS Code installed

code.visualstudio.com — any OS. Open it once before 4 PM.

☐ 3

GitHub account

github.com/signup. Note your username — you will push with it.

☐ 4

Stuck? 3:30 desk

Come at 3:30 to L202. We help with install + login. Main show 4 PM.

SESSION 1 → SESSION 2.1

You shipped a bio page. Now version it.

- Session 1 homework: `index.html` bio + links + CSS, pushed with `git add · commit · push`.
- Last session half the batch got stuck on auth — today we fix that first, then continue.
- Tonight's upgrade: proper Git moves *plus* an agent that edits that same page with you.

Speaker cue: show 2–3 best bio pages, 1 min each. Then: "every change from tonight = one commit".

TERMINAL BASICS · 2 MINUTES — THEN WE START

Lost in terminal? Three commands fix that.

WHERE AM I?

`pwd`

Prints your current folder's full path. Run it whenever you feel lost.

WHAT'S HERE?

`ls`

Lists files in this folder. `ls -a` also shows hidden ones like `.git`.

GO THERE

`cd`

`cd <folder>` moves in. `cd ..` goes up one. Tab completes names — use it.

```
$ try all three right now
pwd # → /Users/<username>/bio — you are here
ls # → index.html … these are your files
cd bio # → step into bio, pwd again to confirm
```

PART 1 · THE MENTAL MODEL — READ THIS ONCE, GIT CLICKS FOREVER

Four layers. Three moves between them.

LAYER 1 · MACHINE

Working files

Your edits.
`git status` sees them

—▸ add

LAYER 2 · ON DEVICE

Staged + history

`commit · rebase`
`git log` lives here

—▸ push / pull

LAYER 3 · YOURS ONLINE

Your GitHub repo

`git push` sends
`git pull` fetches

—▸ fork / PR

LAYER 4 · THEIRS ONLINE

Someone else's repo

Fork it, PR it,
merge it back

LIVE · TYPE WITH US

One loop runs the whole night.

```
$ clone once, loop forever
git clone <url>
git status
git add index.html
git commit -m "describe the change"
git push
git pull # before you start
```

- `clone` once — copies Layer 3/4 down to your machine.
- Loop daily: `status → add → commit → push`.
- `pull` first when collaborating — avoids half your conflicts.

PART 1 · KEEP THIS PHOTO ON YOUR PHONE

The only 8 commands tonight.

SEE

`status · log · diff`

What changed, what shipped, line-by-line how.

SAVE

`add · commit`

Stage files, freeze a snapshot with a message.

SYNC

`clone · push · pull`

Copy down, send up, merge down.

SPLIT

`branch · checkout`

Parallel timelines. Break things safely.

Full sheet: git-scm.com/docs + GitHub Skills. Tonight we demo each one live — no mugging up.

main stays safe. You play on a branch.

```
$ experiment without fear
git branch redesign
git checkout redesign # or: git checkout -b
redesign
```




```
git checkout main → git merge redesign
```

- A branch is just a movable label on a commit.
- Rule: never demo-broken code on `main`.
- Merge = join your playground back into `main`.

● LIVE · TRY IT

`clone` + `diff`: look before you leap.

```
$ copy any repo down
git clone
https://github.com/<username>/<reponame>.git
cd <reponame>
git status # what changed?
git diff # exact lines changed
git log --oneline # what shipped?
```

READ THE DIFF LIKE THIS

`-` removed · `+` added

Red lines left your file, green lines entered. If the diff surprises you, don't `add` yet. `diff` is your pre-commit review.

PART 1 · UNDO — THREE LEVELS OF "OOPS"

Undo small. Rarely nuke.

SOFT UNDO

`git restore file`

Discard edits in one file. File goes back to last commit. Safe.

UNSTAGE

`git reset file`

Un-`add` a file. Edits stay, staging undone. Very safe.

☠ NUKE

`git reset --hard`

Erase ALL uncommitted work. No undo. Demo it once, fear it forever.

Golden rule: `--hard` destroys work permanently. Commit first (`git commit -m "wip"`), then experiment — you can always come back.

You never push to their repo. You PR it.

1

Fork

Copy their repo (Layer 4) to your account (Layer 3). One click on GitHub.

2

Clone + branch

`clone` your fork, make a branch, commit the fix.

3

Push + PR

`push` branch, click "Compare & pull request". Describe *why*.

4

Merge

Maintainer reviews, approves, merges. Your commit lands upstream.

Homework repos, GSoC, internships — every contribution you will ever make follows these 4 steps.

Conflicts aren't errors. They're Git asking you.

```
# Git marks the fight — you pick the winner
<<<<<<< HEAD
background: navy; # yours
=======
background: teal; # theirs
>>>>>>> redesign
$ then: edit → git add → git commit
```

- **Why:** same lines changed in two places. Git can't guess — so it asks.
- **Fix:** open the file, delete markers, keep what looks right.
- **Finish:** `add + commit`. Conflict resolved, history clean.

FIX-IT BLOCK · ~20 MIN · EVERYONE TO GREEN

Half-done? Let's finish it.

Some pushed last time. Some hit `Author identity unknown` or a login wall. We fix both live — then break a merge on purpose so you never fear one again.

● LIVE · 30 SECONDS — FIND YOUR GROUP

Which group are you in?

GROUP A · ALREADY GREEN ✓

`push` worked before

Run `git status` — clean? You skip to clone + conflict demo. Help your neighbour.

GROUP B · STUCK ×

Hit config or login error

`Author identity unknown`? Browser never opened? Follow the next 2 slides exactly.

```
$ 3 checks — paste all three
git --version # must print 2.x
git config --global user.name # blank = Group B, do slide 15
git remote -v # no origin? do slide 17 / 18
```

BLANK NAME = CONFIG MISSING · NO ORIGIN = NOT LINKED YET

● LIVE · FIX 1 — FIRST COMMIT ERROR

First `commit` asks: who are you?

```
$ what you saw last time
git commit -m "first bio"
Author identity unknown → failed
$ fix once per laptop
git config --global user.name "Swarnadip Kar"
git config --global user.email "<email>"
git config --global init.defaultBranch main
git commit -m "first bio" # now works ✓
```

- `user.name` = `"Swarnadip Kar"`. Stamped on every commit.
- `user.email` = `<email>`. Links commits to `<username>`.
- `init.defaultBranch main` = never see `master` warning again.

VERIFY: GIT CONFIG --GLOBAL --LIST · --GLOBAL = ONCE PER LAPTOP

FIX 2 · NO TOKENS — 2026 DEFAULT

First push opens a browser. That's it.

```
$ first push from any folder
git push -u origin main
→ browser opens automatically:
Authorize GitCredentialManager? → Allow
$ back in terminal:
Writing objects… done ✓
git credential.helper # saved — no repeat
```

- `-u` = remember: `main` tracks `origin/main`. Next time just `git push`.
- No token pasting. HTTPS + Credential Manager does OAuth for you.
- Browser didn't open? Copy the device code from terminal → paste at github.com/login/device.

ALLOW IN BROWSER ONCE · CACHED TILL YOU SIGN OUT

● LIVE · PATH A — ONLINE FIRST (EASIEST)

Create online → paste 3 lines.

```
$ github.com → New repo → name: <reponame> → Create
# then GitHub shows "...or push an existing repo":
git remote add origin https://github.com/<username>/<reponame>.git
git branch -M main
git push -u origin main
```

- `remote add origin` = save GitHub's address under name `origin`. Once.
- `branch -M main` = rename current branch to `main`. Matches GitHub.
- `push -u origin main` = send + remember. Triggers browser login once.

Speaker: create repo with NO README for this path. Show the copy button on GitHub. Check: `git remote -v` must show your URL.

● LIVE · PATH B — LOCAL FIRST

`init` here → link → push.

```
$ in your project folder
git init -b main # start history on main
git add index.html # stage
git commit -m "first bio page" # snapshot
git remote add origin https://github.com/<username>/<reponame>.git
git push -u origin main # browser → done ✓
```

- `init -b main` = create `.git` folder. History starts here.
- `add + commit` = you must snapshot *before* first push. Empty push fails.
- Same last 2 lines as Path A — both paths converge here.

CONCEPT · THE LINE EVERYONE PASTES BUT NOBODY EXPLAINS

`remote` is an address book. `origin` is a nickname.

```
$ nicknames for URLs — each line below is ONE command
git remote -v # list: must show fetch + push
git remote add origin https://github.com/<username>/<reponame>.git # save
once
git remote set-url origin https://github.com/<username>/<reponame>.git # fix
URL
git remote rename origin myfork # rarely needed
git remote remove origin # unlink only, files stay
```

- `origin` = default nickname for your GitHub repo. Not magic, just a shortcut.
- `upstream` = what your branch tracks: `main → origin/main`. Set once with `push -u`.
- `-u` = `--set-upstream`. After that, bare `push` / `pull` know where to go.

Speaker: run `git remote -v` live — empty = you skipped Path A/B. `set-url` fixes a pasted wrong URL without re-adding.

Branches: one idea, six commands.

```
$ local vs tracked
git branch # list local, * = current
git branch -a # + remote-tracking: origin/main
git checkout -b redesign # create + switch (classic)
git switch -c redesign # create + switch (new)
git switch main # just switch
git push -u origin redesign # publish + track
git branch -d redesign # delete merged (-D = force)
```

- Local `redesign` = your work. `origin/redesign` = GitHub's copy you track.
- Flow: `checkout -b` (or `switch -c`) → work → `push -u` once → `-d` to clean up.
- Lost? `git branch --show-current` tells you where you are.

● LIVE · SECOND COPY

`clone` it again. `pull` before work.

```
$ new folder / friend's laptop
git clone
https://github.com/<username>/<reponame>.git
cd <reponame>
git pull origin main # update + merge
git log --oneline # same history ✓
```

WHY BOTH?

`clone` once · `pull` always

`clone` = full copy + history, first time only. `pull` = fetch + merge latest before you edit. Pull-first avoids half your conflicts.

● LIVE · WE BREAK IT — STEP 1 OF 3

Branch off. Edit. Hold the push.

1. `git checkout -b redesign` — new playground. Confirm with `git branch`.

2. Edit line 10 in `index.html` → `background: teal;` → `git add index.html` → `git commit -m "redesign: teal hero"`. Do NOT push yet.

3. Stay on `redesign`. Hands off keyboard — we move `main` next.

Speaker: everyone edits the SAME line — that's what forces the conflict. Show `git log --oneline`: redesign is 1 ahead.

LIVE · STEP 2 OF 3 — MOVE MAIN FORWARD

Change `main`. Push it. Now they fight.

```
$ same file, different answer
git checkout main
# edit SAME line 10 → background: navy;
git add index.html
git commit -m "main: navy hero"
git push # main now ahead on GitHub ✓
git checkout redesign
git push -u origin redesign # or: merge → ! rejected
```

- You changed the **same line** two ways: `teal` vs `navy`.
- `main` pushed first, so `redesign` is now behind + diverged.
- Next: `git pull origin main` (or merge) → Git stops and asks you.

● LIVE · STEP 3 OF 3 — THE 30-SEC FIX

Delete markers. Keep the winner. Push.

```
# Git marks the fight in index.html
<<<<<<< HEAD # main: navy
background: navy;
=======
background: teal; # redesign
>>>>>>> redesign
$ fix — 3 moves:
git add index.html # after editing
git commit -m "merge redesign: keep teal"
git push
```

- **1 · Edit:** open file, delete `<<< === >>>`, keep one colour (or mix).
- **VS Code tip:** click *Accept Current / Incoming / Both* above the markers.
- **2 · 3 · Done:** `add + commit + push`. Conflict over, history clean.

CHECKPOINT — ARE YOU GREEN?

Clean tree + graph = done.

```
$ everyone runs this
git status # must say: working tree clean ✓
git log --oneline --graph --all # see merge ✓
git pull # Already up to date ✓
```

STILL RED?

Raise your hand now

`rejected` → you forgot `pull`. `no origin` → redo slide 17/18.
`identity unknown` → redo slide 15. Fix-it desk stays open.

FALLBACK ONLY · YOU SHOULDN'T NEED THIS IN 2026

Browser failed? Tokens still work.

1

Create token

Settings → Developer settings → Fine-grained tokens → Generate. Read + write on your repo. Copy once.

2

Push normal

`git push`. Git prompts for login.

3

Paste token

Username = GitHub username. Password = token, *not* your password.

4

Stays saved

Keychain / Credential Manager remembers it. No prompt? `git config --global credential.helper store`.

Try browser login (slide 16) first. Use this only on lab PCs where the browser pop-up is blocked. Next step when ready: SSH keys.

PART 2 · THE MAIN EVENT · LIVE

Your own agent, editing live.

Same bio page from Session 1. This time you won't hand-write every line — you'll direct an agent that does. Watch, then do it on your laptop.

PART 2 · STEP 1 — THE EDITOR BECOMES AN AGENT

An extension = superpower you install.

WHAT

Extensions add skills

Themes, languages, AI agents — VS Code is plain until you extend it. Extensions live in the left sidebar's square icon.

INSTALL

Kilo Code in 30 sec

Extensions (`Ctrl/Cmd+Shift+X`) → search `Kilo Code` → Install → pin it to the sidebar.

OTHERS EXIST

Kilo is one example

Cline, Roo, Copilot, **OpenCode** (terminal-first) — same idea, different shell. Learn one deeply; switching is easy.

Sidebar for clicks. CLI for speed.

```
$ the agent, in your terminal
npm install -g @kilo/cli # or: brew install kilo
kilo # opens agent in this repo
/add index.html # attach context
```

- Kilo **extension** = buttons + chat beside your code. Start here.
- Kilo **CLI** = same brain in the terminal. Same keys, scriptable.
- Prefer terminals? Try **OpenCode** the same way. Concepts transfer.

AI has no memory. Context is your job.

- Remember `proximity-modular-core.json`? ~975k tokens re-read every turn. Same with your agent.
- Tag files: `@index.html` `@style.css` — never say "my page", show it.
- One task per prompt. Small ask → review → next ask. Be the director, not the typist.

```
# vague (bad)
"make it nice"
# directed (good)
"@index.html @style.css:
make hero responsive,
keep green accent,
no new deps" ✓
```

● LIVE · THE 5-MINUTE UPGRADE — DO IT WITH US

Three prompts. Watch your page grow.

1. `@index.html` — "Add a responsive hero with my name, one-line bio, and my GitHub + LinkedIn links. Keep it single-file."

2. `@index.html @style.css` — "Move styles to style.css, add dark section + green accent like the ALGORYTHM slides, mobile-friendly."

3. "Add `data.json` with my name + interests, and render it into the page with vanilla JS. Explain each change."

Speaker: run prompt 1 fully, review the diff in VS Code, accept, then prompt 2. Always show the diff before accepting.

LIVE · FREE KEY IN 2 MINUTES

Agents need a key. Gemini is the easiest free one.

- **Get it:** open aistudio.google.com/apikey → Get API Key → Create → copy once.
- **Use it:** Kilo Code settings → paste key → pick a Gemini model.
- **Protect it:** keys live in `.env`, and `.env` lives in `.gitignore`. Leaked it? Revoke + regenerate immediately.

```
# never commit secrets
echo ".env" >> .gitignore
cat .gitignore # confirm .env listed
git status # .env must NOT appear
```

PART 2 · ONE KEY TO START — ANY KEY WHEN YOU GROW

Start with Gemini. Judge the rest yourself.

★ START HERE

Gemini

Google AI Studio free key. Generous, easy, good for homework edits.

FAST

Groq

Very fast inference free tier. Great when you iterate a lot.

MANY

OpenRouter :free

One key, many `:free` models. Continuation of Session 1.

ZERO-SETUP

Kilo built-in

Trial credits in Kilo itself. Fine for tonight, quota ends.

Don't trust hype — compare on **Artificial Analysis** (quality / speed / price leaderboards) and OpenRouter, then pick free + good over best + paid.

● LIVE · CLOSE THE LOOP

Agent edited it. Now ship it.

```
$ the loop, again — with agent help
git status # review what agent changed
git diff # read it, don't blind-accept
git add index.html style.css data.json
git commit -m "agent: responsive hero +
data.json"
git push
```

- Every agent change = one commit with a clear message.
- `.env` must never appear in `status`. If it does, stop.
- Refresh your GitHub repo — your upgraded page is proof of work.

HUNGRY? DON'T WANDER — FOLLOW A PATH

One roadmap beats ten tutorials.

PATH

roadmap.sh

Frontend → Backend → Git → DevOps, in order. Pick ONE track this month. (Thanks Bhargav for the find.)

RULE

Docs first, then build

Official docs + one small project beat 20 bookmarked videos. Stuck 15 min → ask in the group.

HABIT

One video a day

Fireship for breadth, 3Blue1Brown for depth, Two-Minute Papers for wonder — from Session 1.

PART 3 · HANDOFF — JAGATH TAKES THE STAGE → THEN HOMEWORK

Map ✓ Ship ✓ Game →

- CTF round 2 with Jagath — same workflow as Session 1 (`file · exiftool · strings · binwalk · LSB`), harder images.
- **Homework:** agent-upgrade your bio page, push ≥2 commits, drop the repo link in the WhatsApp group. Best pages on stage next session.
- Doubts → WhatsApp group. Fill the feedback form — it shapes Session 3.

Speaker notes: recap 2 min (login → Path A/B → conflict fix → agent → push), hand mic to Jagath. Total talk ~70–80 min + CTF block.

SEE YOU IN THE CTF BLOCK — LAPTOPS OPEN